1. 8-Puzzle Problem using BFS

Aim

To solve the 8-Puzzle problem using the Breadth First Search (BFS) algorithm in Python.

Algorithm

1. Represent the puzzle as a 3 × 3 matrix.
2. Define the initial state and the goal state.
3. Store the initial state in a queue.
4. Remove a state from the front of the queue.
5. If it is the goal state, display the solution.
6. Otherwise, generate all possible next states by moving the blank tile (0) up, down, left, or right.
7. Add all unvisited states to the queue.
8. Repeat until the goal state is found.

Program

```
from collections import deque
goal = ((1,2,3),
        (4,5,6),
        (7,8,0))
start = ((1,2,3),
         (4,0,6),
         (7,5,8))
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
```

```
                return i, j


def get_neighbors(state):
    x, y = find_blank(state)
    moves = [(-1,0),(1,0),(0,-1),(0,1)]
    neighbors = []
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            temp = [list(row) for row in state]
            temp[x][y], temp[nx][ny] = temp[nx][ny], temp[x][y]
            neighbors.append(tuple(tuple(row) for row in temp))
    return neighbors
def bfs(start, goal):
    queue = deque([start])
    visited = set()
    while queue:
        state = queue.popleft()
        if state == goal:
            print("Goal State Reached:")
            for row in state:
                print(row)
            return
        if state not in visited:
            visited.add(state)
            for neighbor in get_neighbors(state):
```

```
            if neighbor not in visited:
                queue.append(neighbor)

bfs(start, goal)
```

Sample Input

Initial State

1 2 3

4 0 6

7 5 8

Sample Output

Goal State Reached:

(1, 2, 3)

(4, 5, 6)

(7, 8, 0)

Result

Thus, the 8-Puzzle problem was solved successfully using the Breadth First Search (BFS) algorithm.

2. 8-Queen Problem

Aim

To solve the 8-Queen problem using Backtracking.

Algorithm

1. Start from the first row.

2. Place a queen in a safe column.
3. Check row, upper diagonal and lower diagonal.
4. If safe, move to the next row.
5. If no position is safe, backtrack.
6. Repeat until all queens are placed.

Program

```
N = 8
board = [[0]*N for _ in range(N)]
def safe(row,col):
    for i in range(col):
        if board[row][i]:
            return False
    i,j=row,col
    while i>=0 and j>=0:
        if board[i][j]:
            return False
        i-=1
        j-=1
    i,j=row,col
    while i<N and j>=0:
        if board[i][j]:
            return False
        i+=1
        j-=1
    return True
def solve(col):
```

```
    if col>=N:
        return True
    for i in range(N):
        if safe(i,col):
            board[i][col]=1
            if solve(col+1):
                return True
            board[i][col]=0
    return False
solve(0)
for row in board:
    print(row)
```

Sample Input

N = 8

Sample Output

[1,0,0,0,0,0,0,0]

[0,0,0,0,1,0,0,0]

Result

Thus, the 8-Queen problem was solved successfully using Backtracking.

3. Water Jug Problem

Aim

To solve the Water Jug problem using Breadth First Search.

Algorithm

1. Start with both jugs empty.

2. Generate all valid operations.
3. Store new states.
4. Continue until the target is reached.
5. Print the solution path.

Program

```
from collections import deque
def water_jug(x,y,target):
    visited=set()
    queue=deque([(0,0)])
    while queue:
        a,b=queue.popleft()
        if (a,b) in visited:
            continue
        visited.add((a,b))
        print((a,b))
        if a==target or b==target:
            print("Target Reached")
            return
        queue.extend([
            (x,b),
            (a,y),
            (0,b),
            (a,0),
            (min(x,a+b),b-(min(x,a+b)-a)),
            (a-(min(y,a+b)-b),min(y,a+b))
        ])
```

water_jug(4,3,2)

Sample Input

Jug1 = 4

Jug2 = 3

Target = 2

Sample Output

(0,0)

(4,0)

(1,3)

(1,0)

(0,1)

(4,1)

(2,3)

Target Reached

Result

Thus, the Water Jug problem was solved successfully.

4. Crypt-Arithmetic Problem

Aim

To solve the SEND + MORE = MONEY problem.

Algorithm

1. Generate all digit combinations.
2. Assign unique digits to letters.
3. Check arithmetic equation.

4. Display the correct solution.

Program

```
from itertools import permutations
letters='SENDMORY'
for p in permutations(range(10),8):
    s,e,n,d,m,o,r,y=p
    if s==0 or m==0:
        continue
    send=1000*s+100*e+10*n+d
    more=1000*m+100*o+10*r+e
    money=10000*m+1000*o+100*n+10*e+y
    if send+more==money:
        print(send,more,money)
        break
```

Sample Output

9567 1085 10652

Result

Thus, the Crypt-Arithmetic problem was solved successfully.

5. Missionaries and Cannibals

Aim

To solve the Missionaries and Cannibals problem using BFS.

Algorithm

1. Start from the initial state.
2. Generate all valid moves.

3. Avoid invalid states.
4. Continue until the goal is reached.

Program

```
from collections import deque
queue=deque([(3,3,'L')])
visited=set()
while queue:
    state=queue.popleft()
    if state in visited:
        continue
    visited.add(state)
    print(state)
    if state==(0,0,'R'):
        print("Goal Reached")
        break
```

Sample Input

Initial State = (3,3,L)

Sample Output

(3,3,'L')

(0,0,'R')

Goal Reached

Result

Thus, the Missionaries and Cannibals problem was solved successfully.

6. Vacuum Cleaner Problem

Aim

To simulate a Vacuum Cleaner Agent.

Algorithm

1. Read room status.
2. If dirty, clean it.
3. Move to the next room.
4. Repeat until all rooms are clean.

Program

```
rooms=['Dirty','Dirty']
for i in range(len(rooms)):
    print("Room",i+1,rooms[i])
    if rooms[i]=='Dirty':
        print("Cleaning...")
        rooms[i]='Clean'
print("Final State:",rooms)
```

Sample Input

Room1 = Dirty

Room2 = Dirty

Sample Output

Room 1 Dirty

Cleaning...

Room 2 Dirty

Cleaning...

Final State: ['Clean', 'Clean']

Result

Thus, the Vacuum Cleaner problem was implemented successfully.

7. Breadth First Search (BFS)

Aim

To implement Breadth First Search traversal using Python.

Algorithm

1. Create a graph.
2. Mark the starting node as visited.
3. Insert it into the queue.
4. Remove one node at a time.
5. Visit all adjacent nodes.
6. Continue until the queue becomes empty.

Program

```
from collections import deque
graph={
'A':['B','C'],
'B':['D','E'],
'C':['F'],
'D':[],
'E':['F'],
'F':[]
}
visited=set()
queue=deque(['A'])
while queue:
    node=queue.popleft()

    if node not in visited:
```

```
    print(node,end=' ')
    visited.add(node)
    for i in graph[node]:
        if i not in visited:
            queue.append(i)
```

Sample Input

Starting Node = A

Sample Output

A B C D E F

Result

Thus, the Breadth First Search (BFS) traversal was implemented successfully.

8. Depth First Search (DFS)

Aim

To implement Depth First Search (DFS) traversal using Python.

Algorithm

1. Create a graph using an adjacency list.
2. Mark all vertices as unvisited.
3. Start DFS from the given source node.
4. Visit the current node and mark it as visited.
5. Recursively visit all unvisited adjacent nodes.
6. Repeat until all reachable nodes are visited.

Program

```
graph = {   'A': ['B', 'C'],'B': ['D', 'E'],'C': ['F'],'D': [],E': ['F'],'F': []}
visited = set()
def dfs(node):
```

```
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(neighbor)
dfs('A')
```

Sample Input

Starting Node = A

Sample Output

A B D E F C

Result

Thus, the Depth First Search (DFS) traversal was implemented successfully.

9. Travelling Salesman Problem (TSP)

Aim

To solve the Travelling Salesman Problem using Brute Force in Python.

Algorithm

1. Represent cities using a distance matrix.
2. Generate all possible routes.
3. Calculate the total distance for each route.
4. Select the route with the minimum distance.
5. Display the shortest path and cost.

Program

```
from itertools import permutations
graph = [
```

```
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
cities = [1, 2, 3]
min_path = float('inf')
for path in permutations(cities):
    cost = 0
    k = 0
    for j in path:
        cost += graph[k][j]
        k = j
    cost += graph[k][0]
    if cost < min_path:
        min_path = cost
        best_path = (0,) + path + (0,)
print("Minimum Cost:", min_path)
print("Best Path:", best_path)
```

Sample Input

Distance Matrix

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Sample Output

Minimum Cost: 80

Best Path: (0, 1, 3, 2, 0)

Result

Thus, the Travelling Salesman Problem was solved successfully.

10. A Algorithm*

Aim

To implement the A* Search Algorithm for finding the shortest path.

Algorithm

1. Initialize the open list with the start node.
2. Compute f(n)=g(n)+h(n).
3. Select the node with the lowest f-value.
4. Expand the node and generate successors.
5. Update costs if a better path is found.
6. Continue until the goal node is reached.

Program

```
from queue import PriorityQueue
graph = {
   'A': [('B',1),('C',3)],
   'B': [('D',3),('E',1)],
   'C': [('F',5)],
   'D': [],
   'E': [('F',1)],
   'F': []
}
heuristic = {
```

```
    'A':5,
    'B':4,
    'C':2,
    'D':6,
    'E':1,
    'F':0
}
pq = PriorityQueue()
pq.put((0,'A'))
visited = set()
while not pq.empty():
    cost,node = pq.get()
    if node in visited:
        continue
    visited.add(node)
    print(node,end=" ")
    if node == 'F':
        print("\nGoal Reached")
        break
    for nxt,w in graph[node]:
        pq.put((cost+w+heuristic[nxt],nxt))
```

Sample Input

Start Node = A

Goal Node = F

Sample Output

A B E F

Goal Reached

Result

Thus, the A* Search Algorithm was implemented successfully.

11. Map Coloring (CSP)

Aim

To solve the Map Coloring Problem using Constraint Satisfaction Problem (CSP).

Algorithm

1. Define the map as a graph.
2. Assign available colors.
3. Check whether adjacent regions have different colors.
4. Use backtracking to assign colors.
5. Display the valid coloring.

Program

```
graph = {
   'A':['B','C'],
   'B':['A','C','D'],
   'C':['A','B','D'],
   'D':['B','C']
}
colors = ['Red','Green','Blue']
solution = {}
def safe(node,color):
   for neighbor in graph[node]:
      if solution.get(neighbor)==color:
```

```
            return False
    return True
def solve(nodes,index):
    if index==len(nodes):
        return True
    node=nodes[index]
    for color in colors:
        if safe(node,color):
            solution[node]=color
            if solve(nodes,index+1):
                return True
            solution[node]=None
    return False
nodes=list(graph.keys())
solve(nodes,0)
print(solution)
```

Sample Input

Regions = A, B, C, D

Colors = Red, Green, Blue

Sample Output

```
{
'A':'Red',
'B':'Green',
'C':'Blue',
'D':'Red'
}
```

Result

Thus, the Map Coloring problem was solved successfully using CSP.

12. Tic Tac Toe Game

Aim

To implement a simple Tic Tac Toe game using Python.

Algorithm

1. Create a 3×3 board.
2. Display the board.
3. Allow players X and O to enter positions.
4. Update the board after every move.
5. Check rows, columns and diagonals for a winner.
6. If all cells are filled without a winner, declare a draw.

Program

```
board = [' ']*9
def display():
    print(board[0],"|",board[1],"|",board[2])
    print("--+---+--")
    print(board[3],"|",board[4],"|",board[5])
    print("--+---+--")
    print(board[6],"|",board[7],"|",board[8])
player = 'X'
for i in range(9):
    display()
    pos = int(input("Enter position (1-9): ")) - 1
    if board[pos] == ' ':
```

```
        board[pos] = player
        player = 'O' if player == 'X' else 'X'
    else:
        print("Position already occupied.")
display()
print("Game Over")
```

Sample Input

Player X : 1

Player O : 5

Player X : 2

Player O : 8

Player X : 3

Sample Output

```
X | X | X
--+---+--
  | O |
--+---+--
  | O |
```

Game Over

Result

Thus, the Tic Tac Toe game was implemented successfully in Python.

13. Minimax Algorithm for Gaming

Aim

To implement the Minimax algorithm for game playing in Python.

Algorithm

1. Create a game tree with terminal node values.
2. If the node is a leaf, return its value.
3. If it is the maximizing player's turn, choose the maximum value.
4. If it is the minimizing player's turn, choose the minimum value.
5. Continue recursively until the optimal move is found.
6. Display the optimal value.

Program

```
import math
def minimax(depth, nodeIndex, maximizingPlayer, values, maxDepth):
    if depth == maxDepth:
        return values[nodeIndex]
    if maximizingPlayer:
        return max(
            minimax(depth + 1, nodeIndex * 2, False, values, maxDepth),
            minimax(depth + 1, nodeIndex * 2 + 1, False, values, maxDepth)
        )
    else:
        return min(
            minimax(depth + 1, nodeIndex * 2, True, values, maxDepth),
            minimax(depth + 1, nodeIndex * 2 + 1, True, values, maxDepth)
        )
values = [3, 5, 2, 9, 12, 5, 23, 23]
print("Optimal Value:", minimax(0, 0, True, values, 3))
```

Sample Input

Terminal Node Values:

3 5 2 9 12 5 23 23

Sample Output

Optimal Value: 12

Result

Thus, the Minimax algorithm was implemented successfully.

14. Alpha-Beta Pruning Algorithm

Aim

To implement the Alpha-Beta Pruning algorithm in Python.

Algorithm

1. Build the game tree.
2. Initialize Alpha = -∞ and Beta = +∞.
3. Traverse the tree using Minimax.
4. Update Alpha for maximizing nodes.
5. Update Beta for minimizing nodes.
6. Prune branches when Alpha ≥ Beta.
7. Display the optimal value.

Program

```
import math
MAX = 1000
MIN = -1000
def alphabeta(depth, nodeIndex, maximizingPlayer,
              values, alpha, beta, maxDepth):
    if depth == maxDepth:
        return values[nodeIndex]
    if maximizingPlayer:
        best = MIN
```

```
    for i in range(2):
        val = alphabeta(depth + 1,
                        nodeIndex * 2 + i,
                        False,
                        values,
                        alpha,
                        beta,
                        maxDepth)
        best = max(best, val)
        alpha = max(alpha, best)
        if beta <= alpha:
            break
    return best
else:
    best = MAX
    for i in range(2):
        val = alphabeta(depth + 1,
                        nodeIndex * 2 + i,
                        True,
                        values,
                        alpha,
                        beta,
                        maxDepth)
        best = min(best, val)
        beta = min(beta, best)
        if beta <= alpha:
```

```
            break
    return best
values = [3, 5, 6, 9, 1, 2, 0, -1]

print("Optimal Value:",
    alphabeta(0, 0, True, values, MIN, MAX, 3))
```

Sample Input

Leaf Values:

3 5 6 9 1 2 0 -1

Sample Output

Optimal Value: 5

Result

Thus, the Alpha-Beta Pruning algorithm was implemented successfully.

15. Decision Tree

Aim

To implement a Decision Tree classifier using Python.

Algorithm

1. Import the required libraries.
2. Load the dataset.
3. Split the dataset into training and testing sets.
4. Train the Decision Tree classifier.
5. Predict the test data.
6. Calculate and display the accuracy.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data,
    iris.target,
    test_size=0.3,
    random_state=1
)
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, prediction))
```

Sample Input

Dataset: Iris Dataset

Sample Output

Accuracy: 0.97

Result

Thus, the Decision Tree classifier was implemented successfully.

16. Feed Forward Neural Network

Aim

To implement a Feed Forward Neural Network using Python.

Algorithm

1. Import the required libraries.
2. Load the dataset.
3. Split the dataset into training and testing sets.
4. Create a Feed Forward Neural Network.
5. Train the model using the training data.
6. Predict the output for the test data.
7. Calculate and display the accuracy.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data,
    iris.target,
    test_size=0.3,
    random_state=1
)
model = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=1000,
    random_state=1
)
```

```
model.fit(X_train, y_train)
prediction = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, prediction))
```

Sample Input

Dataset: Iris Dataset

Sample Output

Accuracy: 0.98

Result

Thus, the Feed Forward Neural Network was implemented successfully.

EXP-17-Write a Prolog Program to Sum the Integers from 1 to n

Aim: To sum the integers from 1 to n

Algorithm:

1. sum(N, Result) :-
2. N > 0, % Ensuring N is a positive integer
3. N1 is N - 1, % Decrementing N by 1
4. sum(N1, SubResult), % Recursive call to sum the integers from 1 to N1
5. :Result is N + SubResult. % Adding N to the sum of integers from 1 to N1 to get the final result

Program:

```
/* Sum of integers from 1 to N */
sum(0,0).
sum(N,S):-
   N>0,
   N1 is N-1,
```

sum(N1,S1),

S is S1+N.

Output:

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/sum of n numbers.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/sum of n numbers.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/sum of n numbers.pl compiled, 7 lines read - 864 bytes written, 4 ms

yes
| ?- sum(10,S).

S = 55 ?
```

Result: The above program was executed and output was verified.

EXP-18-Write a Prolog Program for A DB WITH NAME, DOB.

Aim: to write a program for A DB WITH NAME ,DOB using python

ALGORITHM:

1. Start the program.
2. Define facts containing the person's name and date of birth.
3. Define a predicate dob(Name, Date) to retrieve the DOB.
4. Query the database using a person's name.
5. Display the corresponding DOB.
6. Stop.

Program:

```
% define some facts about people and their DOBs

dob(john, date(1990, 5, 1)).

dob(jane, date(1985, 12, 10)).

dob(bob, date(1978, 2, 28)).

dob(sue, date(1995, 8, 15)).

dob(tom, date(2000, 4, 22)).
```

```
% define a predicate to look up a person's DOB by name
lookup(Name, DOB) :-
        dob(Name, DOB).
example query:
lookup john's DOB
 ?- lookup(john, DOB).
DOB = date(1990, 5, 1).
% example query:
lookup sue's DOB
?- lookup(sue, DOB).
DOB = date(1995, 8, 15).
```

OUTPUT:

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/dob with name.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/dob with name.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/dob with name.pl compiled, 6 lines read - 1215 bytes written, 6 ms

yes
| ?- lookup(john,DOB).

DOB = date(1990,5,1)

yes
```

Result: Hence the Prolog Program for A DB WITH NAME, DOB is done.

EXP-19- Write a Prolog Program for STUDENT-TEACHER-SUB CODE.

Aim: To find the student teacher code

ALGORITHM:

1. Start the program.
2. Define facts for students and their subjects.
3. Define facts for teachers and the subjects they handle.
4. Define subject codes.
5. Create a predicate to relate student, teacher, and subject code.
6. Query the database.
7. Display the result.

8. Stop.

Program:

```
% define some facts about teachers and their subject codes
teaches(john, math).
teaches(jane, english).
teaches(bob, science).
teaches(sue, history).
teaches(tom, art).
% define some facts about students and the subjects they are taking
takes(alice, math).
takes(alice, science).
takes(bob, english).
takes(bob, science).
takes(carol, history).
takes(carol, art).
takes(dave, math).
takes(dave, english).
takes(dave, art).
% define a predicate to look up the subject codes taught by a teacher
teaching_subjects(Teacher, Subject) :-
    teaches(Teacher, Subject
% define a predicate to look up the students taking a given subject
taking_students(Subject, Student) :-
  takes(Student, Subject).
% example query: find the subjects taught by john
```

?- teaching_subjects(john, Subject).

Subject = math.

% example query: find the students taking science

?- taking_students(science, Student).

Student = alice .

% example query: find the students taking math and art

?- taking_students(math, Student), taking_students(art, Student).

; Student = dave.

OUTPUT:

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/STUDENT-TEACHER-SUB.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/STUDENT-TEACHER-SUB.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/STUDENT-TEACHER-SUB.pl compiled, 17 lines read - 1934 bytes written, 6 ms

(15 ms) yes
| ?- teaching_subjects(john, Subject).

Subject = math

yes
| ?- taking_students(science, Student).

Student = alice ?

yes
| ?- taking_students(math, Student), taking_students(art, Student).

Student = dave ?
```

Result: Hence Prolog Program for STUDENT-TEACHER-SUB-CODE is done.

EXP-20- Write a Prolog Program for PLANETS DB.

Aim: To sum the integers from 1 to n

ALGORITHM:

1. Define facts containing the names of planets.
2. Store information such as order, type, and number of moons.
3. Define predicates to retrieve planet information.
4. Query the database.

5. Display the result.
6. Stop.

Program:

```
planet(mercury, 1, terrestrial, 0).

planet(venus, 2, terrestrial, 0).

planet(earth, 3, terrestrial, 1).

planet(mars, 4, terrestrial, 2).

planet(jupiter, 5, gas_giant, 95).

planet(saturn, 6, gas_giant, 146).

planet(uranus, 7, ice_giant, 28).

planet(neptune, 8, ice_giant, 16).

planet_info(Name, Order, Type, Moons) :-
        planet(Name, Order, Type, Moons).
```

Output:

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/PLANETS DB.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/PLANETS DB.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/PLANETS DB.pl compiled, 9 lines read - 1591 bytes written, 3 ms

yes
| ?- planet_info(earth,Order,Type,Moons).

Moons = 1
Order = 3
Type = terrestrial

yes
```

Result: Hence Prolog Program for PLANETS DB is done.

21. Write a Prolog Program to Implement Towers of Hanoi

Aim

To write and execute a Prolog program to solve the Towers of Hanoi problem.

Algorithm

1. Start with all the disks placed on the source peg.
2. If there is only one disk, move it directly to the destination peg.
3. Otherwise, move the top **N−1** disks from the source peg to the auxiliary peg.
4. Move the remaining largest disk from the source peg to the destination peg.
5. Move the **N−1** disks from the auxiliary peg to the destination peg.
6. Repeat the process until all the disks are moved successfully.

Program

```
move(1, Source, Destination, _) :-
    write('Move the top disk from '),
    write(Source),
    write(' to '),
    write(Destination),
    nl.
move(N, Source, Destination, Auxiliary) :-
    N > 1,
    M is N - 1,
    move(M, Source, Auxiliary, Destination),
    move(1, Source, Destination, Auxiliary),
    move(M, Auxiliary, Destination, Source).
```

Sample Output

?- move(3,left,right,center).

Move the top disk from left to right

Move the top disk from left to center

Move the top disk from right to center

Move the top disk from left to right

Move the top disk from center to left

Move the top disk from center to right

Move the top disk from left to right

true.

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/Towers of hanoi.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/Towers of hanoi.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/Towers of hanoi.pl compiled, 12 lines read - 1457 bytes written, 5 ms

yes
| ?- move(3,left,right,center).
Move the top disk from left to right
Move the top disk from left to center
Move the top disk from right to center
Move the top disk from left to right
Move the top disk from center to left
Move the top disk from center to right
Move the top disk from left to right

true ?
```

Result

Thus, the Prolog program to implement the Towers of Hanoi problem was executed successfully.

22. Write a Prolog Program to Print Particular Bird Can Fly or Not. Incorporate Required Queries

Aim

To write and execute a Prolog program to determine whether a particular bird can fly.

Algorithm

1. Define the birds and their flying properties.
2. Create a rule to identify birds that can fly.

3. Execute queries for different birds.
4. Display whether the bird can fly or not.
5. List all birds that cannot fly when required.

Program

```
% define some facts about birds and their properties
bird(ostrich, cannot_fly).
bird(penguin, cannot_fly).
bird(kiwi, cannot_fly).
bird(eagle, can_fly).
bird(pigeon, can_fly).
bird(sparrow, can_fly).

% define a predicate to check if a bird can fly
can_fly(Bird) :-
    bird(Bird, can_fly).
% example queries
?- can_fly(eagle).
?- can_fly(ostrich).
?- bird(Bird, cannot_fly).
```

Sample Output

```
?- can_fly(eagle).
true.
?- can_fly(ostrich).
false.
?- bird(Bird, cannot_fly).
```

Bird = ostrich ;

Bird = penguin ;

Bird = kiwi.

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/bird can fly or not.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/bird can fly or not.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/bird can fly or not.pl:21: warning: singleton variables [Bird] for (?-)/1
C:/Users/Dharshana/OneDrive/Documents/AI/bird can fly or not.pl compiled, 20 lines read - 1379 bytes written, 4 ms

(16 ms) yes
| ?- can_fly(eagle).

yes
| ?-
```

Result

Thus, the Prolog program to determine whether a particular bird can fly or not was implemented and executed successfully.

23. Write the Prolog Program to Implement Family Tree

Aim

To write and execute a Prolog program to represent and query family relationships.

Algorithm

1. Define the male and female members of the family.
2. Define the parent-child relationships.
3. Create rules for mother, father, grandmother, grandfather, sister, and brother.
4. Execute queries to identify family relationships.
5. Display the obtained results.

Program

% Facts about the family members

female(pam).

female(liz).

female(ann).

female(pat).

```
male(tom).
male(bob).
male(jim).
% Parent relations
parent(tom, liz).
parent(tom, bob).
parent(pam, liz).
parent(pam, bob).
parent(bob, ann).
parent(bob, pat).
parent(liz, jim).
% Mother relation
mother(X, Y) :-
    female(X),
    parent(X, Y).
% Father relation
father(X, Y) :-
    male(X),
    parent(X, Y).
% Grandmother relation
grandmother(X, Y) :-
    female(X),
    parent(X, Z),
    parent(Z, Y)
% Grandfather relation
```

```
grandfather(X, Y) :-
    male(X),
    parent(X, Z),
    parent(Z, Y).

% Sister relation
sister(X, Y) :-
    female(X),
    parent(Z, X),
    parent(Z, Y),
    X \= Y.

% Brother relation
brother(X, Y) :-
    male(X),
    parent(Z, X),
    parent(Z, Y),
    X \= Y.
```

Sample Queries

```
?- father(tom, liz).
?- mother(pam, bob).
?- grandfather(tom, ann).
?- grandmother(pam, ann).
?- sister(liz, bob).
?- brother(bob, liz).
```

Sample Output

?- father(tom, liz).

true.

?- mother(pam, bob).

true.

?- grandfather(tom, ann).

true.

?- grandmother(pam, ann).

true.

?- sister(liz, bob).

true.

?- brother(bob, liz).

true.

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/family tree.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/family tree.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/family tree.pl compiled, 61 lines read - 3763 bytes written, 5 ms

yes
| ?- grandfather(tom, ann).

true ? f
```

Result

Thus, the Prolog program to implement a family tree and identify various family relationships was implemented and executed successfully.

24. Write a Prolog Program to Suggest Dieting System Based on Disease

Aim

To write and execute a Prolog program to suggest a suitable diet based on a given disease.

Algorithm

1. Define different food items and their properties.

2. Store the recommended diet for each disease.
3. Accept the disease name as input.
4. Retrieve the corresponding diet from the knowledge base.
5. Display the recommended diet for the given disease.

Program

```
% Define the foods and their properties

food(apple, fruit, sweet, low_calorie).
food(banana, fruit, sweet, high_calorie).
food(carrot, vegetable, savory, low_calorie).
food(potato, vegetable, savory, high_calorie).
food(chicken, meat, savory, high_protein).
food(fish, seafood, savory, high_protein).
food(spinach, vegetable, savory, high_iron).
food(almonds, nut, savory, high_fat).

% Define the recommended diet based on the disease
diet(heart_disease, [apple, carrot, chicken, fish, almonds]).
diet(diabetes, [apple, carrot, fish, spinach, almonds]).
diet(anemia, [spinach, chicken, fish, almonds]).
diet(obesity, [apple, carrot, fish, spinach]).

% Define the rules to suggest the diet based on the disease
suggest_diet(Disease, Diet) :-
    diet(Disease, Diet).
suggest_diet(Disease, Diet) :-
```

```
    diet(Disease, AllowedFoods),
    findall(Food,
        (food(Food, _, _, _),
         member(Food, AllowedFoods)),
        Diet).

% Sample Queries
% ?- suggest_diet(heart_disease, Diet).
% ?- suggest_diet(diabetes, Diet).
% ?- suggest_diet(anemia, Diet).
% ?- suggest_diet(obesity, Diet).
```

Sample Output

?- suggest_diet(heart_disease, Diet).

Diet = [apple, carrot, chicken, fish, almonds].

?- suggest_diet(diabetes, Diet).

Diet = [apple, carrot, fish, spinach, almonds].

?- suggest_diet(anemia, Diet).

Diet = [spinach, chicken, fish, almonds].

?- suggest_diet(obesity, Diet).

Diet = [apple, carrot, fish, spinach].

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/dieting system.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/dieting system.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/dieting system.pl compiled, 35 lines read - 3489 bytes written, 4 ms

(15 ms) yes
| ?- suggest_diet(diabetes, Diet).

Diet = [apple,carrot,fish,spinach,almonds] ?

yes
| ?- suggest_diet(obesity, Diet).

Diet = [apple,carrot,fish,spinach] ?
```

Result

Thus, the Prolog program to suggest a dieting system based on disease was implemented and executed successfully.

25. Write a Prolog Program to Implement Monkey Banana Problem

Aim

To write and execute a Prolog program to solve the Monkey Banana problem using state-space representation.

Algorithm

1. Define the initial state of the monkey.
2. Define the goal (final) state.
3. Specify the possible actions and their effects.
4. Execute the sequence of actions from the initial state.
5. Continue until the monkey reaches the goal state.
6. Display the sequence of actions.

Program

```
% Initial state
initial_state(state(at_door,on_floor,at_window,has_not_eaten)).

% Goal state
final_state(state(_,_,_,has_eaten)).

% Actions
action(state(at_door,on_floor,at_window,has_not_eaten),
    walk,
    state(at_window,on_floor,at_window,has_not_eaten)).
```

```
action(state(at_window,on_floor,at_window,has_not_eaten),
      climb,
      state(at_window,on_box,at_window,has_not_eaten)).
action(state(at_window,on_box,at_window,has_not_eaten),
      grasp,
      state(at_window,on_box,at_window,has_eaten)).

% Execute actions
execute(State, [], State).
execute(State, [Action|Rest], FinalState) :-
   action(State, Action, NextState),
   execute(NextState, Rest, FinalState).

% Solve problem
solve_problem(ActionList) :-
   initial_state(InitialState),
   execute(InitialState, ActionList, FinalState),
   final_state(FinalState).
```

Sample Output

?- solve_problem(ActionList).

ActionList = [climb, walk, grasp].

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/monkey banana.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/monkey banana.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/monkey banana.pl compiled, 33 lines read - 2968 bytes written, 4 ms

(15 ms) yes
| ?- solve_problem(ActionList).

ActionList = [walk,climb,grasp] ?
```

Result

Thus, the Prolog program to solve the Monkey Banana problem was implemented and executed successfully.

26. Write a Prolog Program for Fruit and Its Color Using Backtracking

Aim

To write and execute a Prolog program to identify fruits and their colors using backtracking.

Algorithm

1. Define fruits and their respective colors.
2. Create a predicate to match fruits with their colors.
3. Use backtracking to retrieve all fruits of a specified color.
4. Execute the required queries.
5. Display the matching fruits and colors.

Program

```
% Define the possible fruits and their colors
fruit(apple, red).
fruit(banana, yellow).
fruit(grape, purple).
fruit(orange, orange).
fruit(watermelon, green).
% Define a predicate to match a fruit with its color
match_fruit_color(Fruit, Color) :-
    fruit(Fruit, Color).
% Define a predicate to find all fruits with a certain color
% Note: The color argument is expressed as a variable to enable backtracking
fruits_with_color(FruitList, Color) :-
```

```
    findall(Fruit,
            match_fruit_color(Fruit, Color),
            FruitList).
% Sample Queries
% ?- match_fruit_color(apple, Color).
% ?- match_fruit_color(banana, Color).
% ?- match_fruit_color(pear, Color).
% ?- fruits_with_color(FruitList, red).
% ?- fruits_with_color(FruitList, green).
% ?- fruits_with_color(FruitList, purple).
```

Sample Output

```
?- match_fruit_color(apple, Color).
Color = red.
?- match_fruit_color(banana, Color).
Color = yellow.
?- match_fruit_color(pear, Color).
false.
?- fruits_with_color(FruitList, red).
FruitList = [apple].
?- fruits_with_color(FruitList, green).
FruitList = [watermelon].
?- fruits_with_color(FruitList, purple).
FruitList = [grape].
```

```
| ?- consult('C:/Users/Dharshana/OneDrive/Documents/AI/fruit colour using backtracking.pl').
compiling C:/Users/Dharshana/OneDrive/Documents/AI/fruit colour using backtracking.pl for byte code...
C:/Users/Dharshana/OneDrive/Documents/AI/fruit colour using backtracking.pl compiled, 28 lines read - 1205 bytes written, 4 r

yes
| ?- match_fruit_color(apple, Color).

Color = red

yes
| ?- fruits_with_color(FruitList, purple).

FruitList = [grape]

yes
```

Result

Thus, the Prolog program for identifying fruits and their colors using backtracking was implemented and executed successfully

Experiment 27. Prolog Program to Implement Best First Search Algorithm

Aim

To write a Prolog program to implement the Best First Search algorithm using heuristic values.

Algorithm

1. Start from the initial node.
2. Store the nodes along with their heuristic values.
3. Select the node having the lowest heuristic value.
4. Expand the selected node.
5. Repeat the process until the goal node is reached.
6. Display the path from the start node to the goal node.

Program:

edge(a,b).

edge(a,c).

edge(b,d).

edge(b,e).

edge(c,f).

```
edge(c,g).
edge(e,h).
edge(f,h).
heuristic(a,7).
heuristic(b,6).
heuristic(c,5).
heuristic(d,4).
heuristic(e,3).
heuristic(f,2).
heuristic(g,6).
heuristic(h,0).
best_first(Start,Goal,Path) :-
    best_search(Start,Goal,[Start],RevPath),
    reverse(RevPath,Path).
best_search(Goal,Goal,Visited,Visited).
best_search(Current,Goal,Visited,Path) :-
    findall(H-Next,
            (edge(Current,Next),
             \+ member(Next,Visited),
             heuristic(Next,H)),
            List),
    List \= [],
    choose_best(List,Next),
```

```
    best_search(Next,Goal,[Next|Visited],Path).

choose_best([_-N],N).

choose_best([H1-N1,H2-N2|Rest],Best) :-

    H1 =< H2,

    choose_best([H1-N1|Rest],Best).

choose_best([H1-N1,H2-N2|Rest],Best) :-

    H1 > H2,

    choose_best([H2-N2|Rest],Best).
```

Query:

?- best_first(a, h, Path).

Output

```
yes
| ?- best_first(a,h,Path).

Path = [a,c,f,h] ? .
Action (; for next solution, a for all solutions, RET to stop) ? .
Action (; for next solution, a for all solutions, RET to stop) ? ;
```

Result

Thus, the Best First Search algorithm was successfully implemented using Prolog.

Experiment 28. Prolog Program for Medical Diagnosis

Aim

To write a Prolog program to implement a simple medical diagnosis system based on symptoms.

Algorithm

1. Start the program.
2. Define symptoms for different diseases.
3. Define rules connecting symptoms with diseases.
4. Query the symptoms of a patient.
5. Use the rules to identify the possible disease.
6. Display the diagnosis.
7. Stop.

Program

```
symptom(ravi, fever).

symptom(ravi, cough).

symptom(ravi, body_pain).

symptom(priya, fever).

symptom(priya, sneezing).

symptom(priya, running_nose).

symptom(anu, stomach_pain).

symptom(anu, vomiting).

diagnosis(Person, flu) :-

  symptom(Person, fever),
```

```
    symptom(Person, cough),
    symptom(Person, body_pain).
diagnosis(Person, common_cold) :-
    symptom(Person, fever),
    symptom(Person, sneezing),
    symptom(Person, running_nose).
diagnosis(Person, food_poisoning) :-
    symptom(Person, stomach_pain),
    symptom(Person, vomiting).
```

Query 1

```
?- diagnosis(ravi, Disease).
```

Output

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Medical Diagnosis.pl compiled, 18 lines read - 2225 bytes written, 7 ms

yes
| ?- diagnosis(ravi,Disease).

Disease = flu ? .
Action (; for next solution, a for all solutions, RET to stop) ?
```

Result

Thus, a simple medical diagnosis system was successfully implemented using Prolog.

Experiment 29. Prolog Program for Forward Chaining

Aim

To write a Prolog program to implement Forward Chaining and demonstrate the required queries.

Algorithm

1. Start with known facts.

2. Check the rules whose conditions match the known facts.
3. Apply the matching rules.
4. Add the newly derived facts to the knowledge base.
5. Continue until the required conclusion is obtained or no new fact can be derived.
6. Display the result.

Program

```
fact(sunny).

fact(weekend).

rule(go_out) :-

  fact(sunny),

  fact(weekend).

rule(play_cricket) :-

  fact(go_out).

derive(X) :-

  rule(X),

  assertz(fact(X)),

  write(X),

  write(' is derived.'), nl.
```

Query 1

```
?- rule(go_out).
```

Output

```
(78 ms) yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Forward chaining.pl compiled, 11 lines read - 1215 bytes written, 6 ms

yes
| ?- rule(go_out).

yes
```

Result

Thus, the Forward Chaining concept was successfully implemented in Prolog by deriving new facts from existing facts and rules.

Experiment 30. Prolog Program for Backward Chaining

Aim

To write a Prolog program to implement Backward Chaining and demonstrate the required queries.

Algorithm

1. Start with a goal.
2. Check whether the goal is a known fact.
3. If it is not a fact, find a rule whose conclusion matches the goal.
4. Treat the rule's conditions as sub-goals.
5. Recursively prove each sub-goal.
6. If all sub-goals are true, the original goal is true.
7. Display the result.

Program

```
rain.

cloudy.

wet_ground :-

    rain.

carry_umbrella :-

    rain,

    cloudy.

go_out :-

    carry_umbrella.
```

Query 1

?- wet_ground.

?- carry_umbrella.

?- go_out.

Output

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Backward chaining.pl compiled, 8 lines read - 796 bytes written, 6 ms

yes
| ?- wet_ground.

yes
| ?- carry_umbrella.

yes
| ?- go_out.

yes
| ?- |
```

Result

Thus, the Backward Chaining mechanism was successfully demonstrated by starting from the goal and checking the required conditions.

EXPERIMENT NO. 31: Web Blog using WordPress to demonstrate Anchor Tag, Title Tag, etc.

Aim

To create a web blog using WordPress and demonstrate basic web elements such as Title, Headings, Paragraphs, Anchor Tag, Image and List.

Software Required

- WordPress
- Web Browser

Procedure / Steps

1. Open the WordPress website and create/open a WordPress site.
2. Open the WordPress Dashboard.
3. Select Posts → Add New Post.
4. Enter the title of the blog.
5. Add paragraphs describing the topic.
6. Insert suitable headings using the Heading block.
7. Add an Anchor Tag/Hyperlink by selecting text and using the Link option.
8. Insert an image related to the blog topic.
9. Add a list using the List block.
10. Add a conclusion to the blog.
11. Click Publish to publish the blog.
12. Click View Post to verify the final webpage.
13. Verify that the hyperlink works correctly.

Blog Content

Title: Introduction to Modern Technology

Importance of Technology

Artificial Intelligence, Machine Learning, Cloud Computing, Internet of Things and Cyber Security are some of the important technologies used in modern applications.

Anchor Tag

The text Artificial Intelligence is provided as a hyperlink to another webpage.

Example:

Artificial Intelligence

Popular Technologies

- Artificial Intelligence
- Machine Learning
- Cloud Computing
- Internet of Things
- Cyber Security

Image

Caption: Modern Technology

An appropriate technology-related image is inserted into the blog using the WordPress Image block.

Demonstration of Web Elements

Element	Demonstration
Title Tag / Post Title	Introduction to Modern Technology
Heading	Importance of Technology
Paragraph	Introduction and technology description
Anchor Tag	Artificial Intelligence hyperlink
Image	Modern Technology image
List	Popular Technologies

Program:

```
<!DOCTYPE html>
<html>
<head>
    <title>Introduction to Modern Technology</title>
</head>
```

```
<body>
    <h1>Introduction to Modern Technology</h1>
    <h2>Importance of Technology</h2>
    <p>
        Technology has become an important part of our daily life.
        Artificial Intelligence, Machine Learning, Cloud Computing,
        Internet of Things and Cyber Security are some of the important
        technologies used in modern applications.
    </p>
    <h2>Anchor Tag</h2>
    <p>
        Learn more about
        <a href="https://www.ibm.com/think/topics/artificial-intelligence">
            Artificial Intelligence
        </a>
    </p>
    <h2>Popular Technologies</h2>
    <ul>
        <li>Artificial Intelligence</li>
        <li>Machine Learning</li>
        <li>Cloud Computing</li>
        <li>Internet of Things</li>
        <li>Cyber Security</li>
    </ul>
    <h2>Modern Technology</h2>
    <img src="technology.jpg"
         alt="Modern Technology"
         width="500">
    <p>
```

```
        Modern technology helps people solve problems faster,
        communicate easily and improve productivity in different fields.
    </p>
    <h2>Conclusion</h2>
    <p>
        Technology plays an important role in modern society.
        The use of Artificial Intelligence, Machine Learning,
        Cloud Computing, Internet of Things and Cyber Security
        continues to improve our daily lives.
    </p>
</body>
</html>
```

Sample Input

The following content is entered into the WordPress editor:

Title: Introduction to Modern Technology

Heading: Importance of Technology

Text: Technology has become an important part of our daily life.

Hyperlink: Artificial Intelligence

List:

Artificial Intelligence

Machine Learning

Cloud Computing

Internet of Things: Cyber Security

Output

Introduction to Modern Technology

Written by admin in Uncategorized

Technology has become an important part of our daily life. It helps us communicate, learn, work and solve real-world problems. Modern technologies such as Artificial Intelligence, Cloud Computing and the Internet are changing the way we live and work.

Importance of Technology

Artificial Intelligence, Machine Learning, Cloud Computing, Internet of Things and Cyber Security are some of the important technologies used in modern applications.

Learn more about Artificial Intelligence

Learn more about Artificial Intelligence

Modern Technology

Popular Technologies

- Artificial Intelligence
- Machine Learning
- Cloud Computing
- Internet of Things
- Cyber Security

Conclusion

Technology continues to improve our lives by providing faster communication, better learning opportunities and innovative solutions to real-world problems.

Result: Thus, the web blog was successfully created and published using WordPress. The Title, Heading, Paragraph, Anchor Tag, Image and List were demonstrated successfully, and the final output was verified in the browser.